

K-Means vs PAM Clustering in R: Partitional Methods Explained

The Concept of Clustering

Clustering is an **unsupervised learning** technique that organizes observations into groups (clusters) — with no predefined target variable. The algorithm must discover structure on its own, guided only by a notion of similarity.

Intra-cluster

Maximize **similarity** among elements within the same group.

Inter-cluster

Minimize **similarity** between elements belonging to different groups.

Similarity Definition

Varies by method: **distance**, **density**, connectivity, and others — each shaping a different view of structure.

Key Characteristics of Clustering

Building on that definition, three core characteristics shape how clustering algorithms work — and what sets them apart from one another.

No Target Variable

The algorithm learns structure **exclusively from the input features**. There are no labels, no external supervision, and no response variable — the model discovers patterns entirely on its own.

Similarity Measures

Each method imposes a different notion of proximity: **K-means** uses Euclidean distance to a centroid, while **DBSCAN** uses regional density to define what it means to be "close."

Latent Structure

The goal is to **reveal hidden natural groupings** that emerge from the data's distribution — patterns that exist but were never explicitly labeled.

We will explore each of these characteristics in depth through practical, hands-on examples implemented in **R**.

Practical Guide: Clustering with daltoolbox in R

With the core concepts established, it's time to see clustering in action. In this section, we dive into hands-on R examples using the `daltoolbox` package — working through methods in a carefully structured sequence of increasing complexity.

The Methods We'll Explore

Partitional Methods

We begin with K-means and PAM — algorithms that partition data into k groups by minimizing intra-cluster variance. A natural first step for understanding cluster geometry.

Density-Based Structure

Building on partitional methods, we examine DBSCAN — an approach that defines clusters through regional density and automatically flags noise points that don't belong to any group.

Model Selection

Finally, we use `clu_tune` to systematically search the hyperparameter space and select the most robust, well-defined clustering solution for a given dataset.

Partitional Methods

K-Means is the most foundational partitional clustering algorithm. It is simple to implement, computationally efficient, and highly interpretable — making it the standard starting point for any unsupervised analysis. The method partitions data into **k non-overlapping groups** by iteratively minimizing intra-cluster variance, with no target variable required during fitting.

How the algorithm works

Initialize the centroids

Randomly selects **k initial points** in the feature space as the starting centroid of each cluster. The choice of initialization can influence the final solution.

Assign points to the nearest centroid

Each observation is assigned to the cluster whose centroid minimizes the **Euclidean distance**, forming k disjoint partitions of the dataset.

Update the centroids

Each centroid is recalculated as the **arithmetic mean** of all observations currently assigned to that cluster, shifting it toward the cluster's center of mass.

Repeat until convergence

Steps 2 and 3 are repeated until cluster assignments **no longer change** between iterations, indicating the algorithm has reached a stable local optimum.

Key Properties

Objective Function

Minimizes the **Within-Cluster Sum of Squares (WCSS)**:
$$WCSS = \sum_k \sum_i \|x_i - \mu_k\|^2$$
where μ_k is the centroid of cluster k. Sensitive to scale — normalization is recommended.

Complexity

Time complexity is **$O(n \cdot k \cdot i \cdot d)$** , where:
n = number of observations
k = number of clusters
i = number of iterations
d = number of dimensions
Generally scales well for moderate datasets.

Assumptions

Clusters are assumed to be **convex, compact, and roughly equal in size**. The algorithm uses spherical geometry (Euclidean distance), so it performs poorly on elongated, irregular, or density-varying cluster shapes.

 **Warning:** Results depend on the **random initialization** of centroids — different runs may produce different clusters. Additionally, the value of **k must be defined before training** and is not learned automatically by the algorithm.

Common Errors – K-Means

Wrong Choice of k

Defining k mechanically (e.g., always $k=3$) without considering the analytical purpose leads to meaningless clusters. Use the elbow method or indices such as silhouette to guide your choice.

Lack of Normalization

Features on different scales distort Euclidean distances, causing the algorithm to favor variables with larger magnitudes. Always normalize (Min-Max or Z-score) before applying K-Means.

Sensitivity to Outliers

Outliers shift centroids disproportionately, corrupting the clusters. Identify and handle outliers before clustering, or consider more robust algorithms such as K-Medoids.

Random Initialization

Random initial centroids may converge to local minima, producing unstable results. Use multiple initializations (`n_init`) or the **K-Means++** algorithm to reduce this risk.

 **Warning:** Real labels (`Species`) are used *exclusively after* fitting, for pedagogical evaluation — **never during training**. Using them beforehand would constitute data leakage.

cluster_kmeans – Configuration and Fitting

R Code

```
# install and load the package
# install.packages("daltoolbox")
library(daltoolbox)

# loading the iris dataset - 150 observations, 4 numeric variables
data(iris)

# method configuration with k=3 - one cluster per expected species
model <- cluster_kmeans(k=3)

# model fitting and labeling
model <- fit(model, iris[,1:4])
clu <- cluster(model, iris[,1:4])

# distribution of points per group
table(clu)
## clu
## 1 2 3
## 96 33 21
```

Reading the code

- `cluster_kmeans(k=3)` — configures the model with 3 clusters (one per expected species)
- `fit(model, iris[,1:4])` — trains the model on the 4 numeric features (no labels used)
- `cluster(model, iris[,1:4])` — assigns each observation to a cluster
- `table(clu)` — shows the distribution of points across the 3 clusters

❏ The value of `k=3` was chosen to match the known number of species — in practice, `k` must be determined analytically (e.g., elbow method, silhouette index).

cluster_kmeans – Evaluation and Normalization

Key Concepts

External evaluation (pedagogical only)

After clustering, we can compare the resulting clusters with known labels (such as `iris$Species`) to interpret group quality. This is done *after* training — the actual labels **never participate** in the clustering process. The `evaluate()` function computes cluster entropy: the lower the value, the purer the groups.

Why normalize?

K-means uses Euclidean distance. If one variable has a much larger scale (e.g., centimeters vs. millimeters), it dominates the distance calculation and distorts the clusters.

Min-max transformation places all variables on the same scale [0, 1], giving equal weight to each dimension.

Code – Evaluation

```
# --- External evaluation (post-clustering) ---  
# The actual labels are used only for interpretation,  
# they were not used during training.  
eval <- evaluate(model, clu, iris$Species)  
eval  
## $clustering_entropy  
## [1] 0.7375436  
## high entropy = less pure groups
```

 **Practical rule:** Always normalize before applying K-means when variables have different scales. The entropy dropping from 0.737 to 0.418 illustrates this gain.

Code – Normalization & Re-evaluation

```
# --- Min-max normalization ---  
# Places all variables on the [0, 1] scale  
iris_mm <- transform(  
  fit(minmax(), iris), iris)  
  
# Re-fitting and labeling  
model <- fit(model, iris_mm[,1:4])  
clu <- cluster(model, iris_mm[,1:4])  
table(clu)  
## 1 2 3  
## 39 50 61  
## more balanced distribution  
  
# Evaluation after normalization  
eval2 <- evaluate(model, clu, iris_mm$Species)  
## $clustering_entropy  
## [1] 0.4177655  
## lower entropy = purer groups
```

K-Means Results: Before and After Normalization

The following comparison demonstrates the **practical impact of feature scaling** on K-means clustering — showing how Min-Max normalization leads to more balanced and purer cluster assignments.

Without Normalization, k=3

Cluster Distribution: **96, 33, 21**

Clustering Entropy: **0.7375**

With Min-Max Normalization, k=3

Cluster Distribution: **39, 50, 61**

Clustering Entropy: **0.4178**

- ❏ **Why normalization is critical for K-means:** K-means relies on Euclidean distance — variables with larger scales dominate the distance calculation and skew cluster boundaries. Here, entropy dropped from **0.7375 to 0.4178** after normalization, producing more balanced and well-defined clusters. Always normalize before applying K-means when features have different scales.

cluster_pam – Partitioning Around Medoids

PAM is a robust alternative to K-Means that shares the same partitional logic — divide data into **k** non-overlapping clusters — but replaces computed centroids with **medoids**: actual data points that minimize total dissimilarity to all other cluster members. This makes PAM more **interpretable** and significantly more **resistant to outliers**, at the cost of higher computational complexity.

Medoids vs. Centroids

Centroid (K-Means)

Arithmetic mean of all points in the cluster. May **not correspond to any real observation** in the dataset. **Sensitive to outliers** — extreme values shift the mean, distorting the cluster center.

Medoid (PAM)

The actual data point that **minimizes total dissimilarity** to all other members of the cluster. Always a **real, observable data point**. **Robust to outliers** — extreme values have less influence on the selection of the most representative point.

Key Properties

Objective Function

Minimizes **Total Within-Cluster Dissimilarity** — the sum of distances from each point to its medoid. Compatible with **any distance metric**, not just Euclidean.

Complexity

$O(k \cdot (n-k)^2)$ per iteration — more computationally expensive than K-Means, but delivers more stable and robust cluster assignments.

When to Use

Noisy data or presence of **outliers**; when **interpretability** of the cluster center is essential; when features have different distance semantics.

Results on Iris Dataset

Cluster Distribution

clu: 1 2 3
50 62 38

Clustering Entropy

0.3939 — competitive with normalized K-Means, achieved **without any prior scale preprocessing**.

cluster_pam – Partitioning Around Medoids

What is PAM?

PAM (Partitioning Around Medoids) is a clustering algorithm closely related to K-means, but with a fundamental distinction: rather than using centroid vectors computed as the mean of cluster points, PAM selects **medoids** — actual data points from the dataset — as cluster centers.

- **Similarity to K-means:** requires the number of clusters `k` as a parameter and follows the same fit-and-label workflow
- **Medoids vs. Centroids:** by anchoring clusters to real data points, PAM is **more robust to outliers**, since extreme values cannot artificially shift the cluster center
- **When to use:** datasets with noise, outliers, or whenever interpretability of the cluster center is essential

Step-by-step

- Initialize PAM with `cluster_pam(k=3)` — the same interface as K-means
- Fitting and labeling follow the same pipeline: `fit()` → `cluster()`
- Cluster distribution: 50, 62, and 38 points — cluster 1 is perfectly pure (entropy = 0)
- Clustering entropy: **0.394** — lower than K-means without normalization

📌 **Key concept:** PAM uses medoids — real data points — instead of computed centroids, making it inherently more robust to outliers. Reference: Kaufman & Rousseeuw (1990).

```
# install.packages("daltoolbox")
library(daltoolbox)

data(iris)

# configure PAM with k=3 clusters
# (same interface as K-means)
model <- cluster_pam(k=3)

# fit the model to the data
model <- fit(model, iris[,1:4])
```

```
# assign cluster labels
clu <- cluster(model, iris[,1:4])

# cluster size distribution
table(clu)
## clu
## 1 2 3
## 50 62 38

# external evaluation (clustering entropy)
eval <- evaluate(model, clu, iris$Species)
## $clustering_entropy
## [1] 0.3938863
```

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

